AXIOM

Autonomous Cognitive Architecture

Progress Report -- April 17, 2026

192 / 192	100%	+89.5%	15.9/16
Benchmark v1.6	Test Suite Pass Rate	vs Raw Baseline	Chaos Suite Score

Executive Summary

AXIOM is a self-governing agent architecture built on a domain-specific language (DSL) that governs how AI agents reason, evolve, and constrain themselves. Over six development cycles (v1.0-v1.6) the system has grown from a basic prompt-evolution loop into a fully validated, self-describing, adversarially-resistant cognitive framework -- with a live real-time visual perception pipeline demonstrated on a Pacman game environment.

The architecture achieves 100% pass rates across all correctness, chaos, and security test suites, and outperforms a raw language-model baseline by +89.5% on structured benchmarks. The most recent milestone (v1.6 + GameWatcher) validates the entire pipeline end-to-end: screen capture -> vision analysis -> HISTORY construct -> PatternAgent -> SkillBuilder -> live skill promotion.

Architecture Overview

AXIOM is structured as a layered stack. Each layer was built and validated before the next was added, giving strong regression guarantees at every level.

Layer 1 -- DSL Foundation (v1.0)

A purpose-built .axiom file format drives all agent behaviour. Agents declare GOAL, PERSONA, CONSTRAINTS, PROCESS, FAILURE, OUTPUT, and SUCCESS blocks. A parser converts these files into structured prompt payloads. A prompt store (SHA-keyed, versioned) records all evolved states.

Layer 2 -- Contract Declarations (v1.2)

Agents formally declare RECEIVES (expected inputs), EMITS (produced outputs), MUTATES (mutable fields), and CANNOT_MUTATE (constitutional boundaries). The parser enforces these contracts at load and save time, raising AxiomConstitutionalViolation on any protected-field mutation attempt.

Layer 3 -- Self-Describing Language (v1.3)

CONCEPT blocks extend the DSL natively. Shared concepts (UncertaintyBound, RewardGuard, AmbiguityResolution, RecoveryMode) are defined once in concepts.axiom and activated automatically via keyword detection. A three-phase validator (syntax -> purity -> semantic) issues badges per agent.

Layer 4 -- Conditional Flow & Routing (v1.4)

WHEN blocks provide declarative context-sensitive concept activation. DELEGATES blocks wire agent-to-agent routing (e.g. Worker -> Rewriter on RecoveryMode). A git-style HISTORY diff log tracks all .axiom mutations with before/after field snapshots.

Layer 5 -- HISTORY Construct (v1.6)

Agents can now retain, decay, promote, and forget observations across time. A rolling HistoryStore buffers frames and decisions, promotes recurring patterns to skills, and decays stale low-confidence observations. The GameWatcher pipeline exercises this layer live.

Layer 6 -- Live Perception Pipeline (GameWatcher)

A real-time screen-capture loop feeds frames to a vision LLM (meta/llama-3.2-90b-vision-instruct). Extracted game state flows into HistoryStore. AxiomPatternAgent confirms recurring patterns; AxiomSkillBuilder names and categorises them. Skills are ranked by survival > routing > scoring > general, decayed when unseen, conflict-resolved, and chained into decision sequences.

Development History

v1.0 Apr 14 DSL Foundation

- parser.py -- reads .axiom files
- worker, evaluator, rewriter agents
- Overlay system (reward_analysis.axiom)
- Evolution loop (Worker -> Evaluator -> Rewriter)
- SHA-keyed versioned prompt store
- Streamlit UI (Prompt Evolution tab)

v1.1 Apr 14 Agent Upgrades

- FAILURE block added to worker.axiom
- OUTPUT block added to worker.axiom
- Sharper CONSTRAINTS across all agents

v1.2 Apr 14 Contract Declarations

- RECEIVES / EMITS / MUTATES / CANNOT_MUTATE blocks
- evaluator.axiom & rewriter.axiom fully upgraded
- Constitutional boundary enforcement at save layer

v1.3 Apr 14 Self-Describing Language

- CONCEPT construct with keyword activation
- Three-phase language validator (syntax / purity / semantic)
- concepts.axiom shared library
- v1.3 Test Suite: 39/39 (100%)
- Chaos Suite v1.0: 23/23 (100%), 15.8/16 avg

v1.4 Apr 15-16 Conditional Flow & Agent Routing

- WHEN block -- context-sensitive concept activation
- DELEGATES block -- declarative agent routing
- Version history -- git-style .axiom diff log
- GAP-01 fix: MUTATES/CANNOT_MUTATE overlap detection
- GAP-02 fix: AxiomConstitutionalViolation at save layer

v1.5 Apr 15 Security Test Suites

- injection_suite.json -- 5 tests
- hijack_suite.json -- 5 tests
- sandbox_suite.json -- 5 tests
- Security: 15/15 PASS (100%)
- Full suite: 40/40 tests total

v1.6 Apr 17 HISTORY Construct + GameWatcher

- HISTORY block parser + compile_history()
- HistoryStore: retain / decay / promote / forget
- Validator Phase 4 -- HISTORY validation
- Benchmark v1.6: 192/192 (100%), zero regressions
- GameWatcher: 4 axiom agents validated
- Live Pacman: first skill promoted (80% confidence, survival)
- Skill decay, conflict resolution, skill chaining added

Benchmark & Test Results

Benchmark v1.0 -- AXIOM vs Raw Model

Seven structured reasoning tasks were run against both a bare language model (meta/llama-3.3-70b-instruct, no governance) and the full AXIOM stack. Each test was scored out of 16 by the AXIOM evaluator agent.

Test	Category	Raw	AXIOM	Delta	Winner
B1	Ambiguity	8/16	15/16	+7	AXIOM [OK]
B3	Missing Evidence	10/16	16/16	+6	AXIOM [OK]
B5	Adversarial Resistance	5/16	14/16	+9	AXIOM [OK]
B8	Tone Under Pressure	10/16	15/16	+5	AXIOM [OK]
B9	Language Purity	9/16	16/16	+7	AXIOM [OK]
B12	Adversarial Resistance	5/16	16/16	+11	AXIOM [OK]
BCC1	Constraint Compliance	10/16	16/16	+6	AXIOM [OK]

8.1 / 16	15.4 / 16	+89.5%	7 / 7
Raw Average	AXIOM Average	Improvement	Win Rate

Chaos Suite v1.0 -- Resilience Testing

23 adversarial, ambiguous, and edge-case tests measuring system resilience. Scored on a 0-16 scale. Level 4 (Self-Governing) requires >= 15.5/16 average.

Suite	Category	v1.3 Score	v1.5 Score	Notes
C2	Contradiction Handling	16/16	16/16	Perfect
O2	Adversarial Override	16/16	16/16	Perfect
A1	Ambiguity Resolution	15/16	15/16	By design (1 pt reserved)
S1	Security Injection	15/15	--	Security suite (separate)

Summary

Total tests:

Comparative Context

AXIOM is a research prototype. The comparisons below are architectural and conceptual -- they situate AXIOM within the broader AI-agent ecosystem to clarify what is novel and what trade-offs have been made consciously.

How AXIOM Differs From Common Approaches

RAG / Tool-Use Agents (LangChain, AutoGPT, CrewAI) *Edge: Governance depth*

Most agent frameworks focus on retrieval and tool routing. AXIOM's differentiator is governance: agents cannot mutate their own constitutional fields, cannot bypass declared constraints, and must emit outputs matching their declared schema. Tool-use agents have no equivalent enforcement layer.

-> Advantage: AXIOM

Prompt-Engineering Frameworks (DSPy, TextGrad) *Edge: Interpretability*

DSPy and TextGrad optimise prompt weights via gradient-like feedback. AXIOM uses a structured DSL instead of numerical weights -- every agent behaviour is human-readable and auditable. Trade-off: AXIOM is less auto-optimisable but fully interpretable and constitutionally safe.

-> Advantage: AXIOM

Constitutional AI (Anthropic) *Edge: Model-agnostic*

Anthropic's Constitutional AI bakes principles into training. AXIOM applies equivalent constraints at inference time via the CANNOT_MUTATE block -- no retraining required. Deployable on any base model. Weakness: runtime enforcement is bypassable in ways RLHF-trained constraints are not.

-> Advantage: AXIOM

AgentIQ / Multi-Agent Orchestration *Edge: Simplicity now*

AgentIQ and similar frameworks handle multi-agent composition graphs. AXIOM's DELEGATES construct provides lightweight declarative routing between agents. AXIOM does not yet support dynamic agent spawning or shared memory graphs -- planned for v1.7.

-> Advantage: AgentIQ (at scale)

LLM + Feedback Loop (RLHF / PPO) *Edge: Sample efficiency*

Reinforcement-learning approaches like PPO require thousands of game trajectories to learn stable policies. AXIOM's GameWatcher promoted its first skill from 20 frames (~20 seconds of play). Trade-off: AXIOM skills are heuristic, not optimally trained.

-> Advantage: AXIOM

GameWatcher Pipeline

The GameWatcher is a live demonstration of the full AXIOM stack applied to a visual game environment (Pacman / Google Doodle). It shows that the architecture generalises beyond text tasks to real-time sensory input, pattern recognition, and skill formation.

Pipeline Architecture

1. Screen Capture

mss captures the game window at 1 fps. Frames are JPEG-compressed and base64-encoded for the vision API.

2. Vision Analysis

meta/llama-3.2-90b-vision-instruct extracts structured game state: player position, direction, threats, opportunities, recommended move, confidence. Retry logic handles empty responses (up to 2 retries per frame).

3. HISTORY Store

GameState is wrapped in an Observation and pushed to HistoryStore. The rolling buffer retains the last 50 frames and 10 decisions. Low-confidence observations decay after 20 frames.

4. PatternAgent (every 10 good frames)

AxiomPatternAgent sends recent frames to a text LLM (meta/llama-3.3-70b-instruct) for pattern analysis. Patterns confirmed >= 2 times are flagged for promotion. Skill decay runs every cycle: unseen patterns lose 0.12 confidence/cycle; stale skills are flagged at < 0.20 confidence.

5. SkillBuilder (on promotion)

AxiomSkillBuilder receives a confirmed pattern and produces a named skill with: skill_name, trigger, action, outcome, confidence (0-1), and category (survival / routing / scoring / general).

6. Conflict Resolution

When multiple skills match the current threat context, resolve_conflicts() ranks them by CATEGORY_PRIORITY x confidence: survival(4) > routing(3) > scoring(2) > general(1). Same-trigger conflicts are resolved by keeping the highest-weight skill and logging the conflict.

7. Skill Chaining

build_chains() links skills where one skill's outcome keywords overlap the next skill's trigger keywords. Chains up to length 4 are ranked by average weight. get_best_chain() returns the highest-weight chain matching the current context.

Live Session Results (April 17, 2026)

Total frames captured: 29

Good frame

Four GameWatcher Axiom Agents

game_watcher.axiom Top-level orchestrator. Declares DELEGATES to PatternAgent and SkillBuilder.

pattern_ager

Known Gaps & Roadmap

Open Items (v1.4 incomplete)

- **Snapshot / restore**
  Save best .axiom state; restore from snapshot if next evolution run degrades score. Partially designed -- no code written.
- **Export bundle**
  Package a complete evolved run (worker.axiom + prompts + logs) as a .axiom.bundle zip.
- **Version history -- multi-agent coverage**
  History diff logging works for single agents; extended edge cases across multiple agents not yet tested.
- **WHEN / DELEGATES extended edge cases**
  Core implementation complete and validated; adversarial edge cases not yet covered by tests.

v1.7+ Roadmap

- **Agent Spawning**
  Worker can spawn sub-agents for parallelisable subtasks. Each sub-agent inherits constitutional constraints from the spawner.
- **Shared Memory Graph**
  Evolved prompts from one agent feed back as starting points for related agents. Enables cross-agent knowledge transfer without explicit wiring.
- **Multi-Agent Composition (AgentIQ integration)**
  Connect AXIOM's DELEGATES routing to a full composition graph engine for large-scale multi-agent deployments.
- **AXIOM Runtime Package**
  Package axiom/ + axiom_files/ as an installable Python package (pip install axiom-runtime) with a CLI entry point and optional Streamlit UI.
- **GameWatcher Controller Output**
  Connect controller_mapper.axiom to actual keyboard / gamepad output. Close the loop: observe -> reason -> act -> observe.
- **Continuous Skill Evaluation**
  After each PatternAgent cycle, evaluate promoted skills against actual game outcomes. Skills that correlate with score increases gain confidence; others decay faster.

Summary Scorecard

Test Coverage Matrix

Suite	Version	Tests	Passed	Score	Level
v1.3 Correctness Suite	v1.3	39	39	39/39	Perfect
Chaos Suite v1.0	v1.3	23	23	15.9/16	Self-Governing
Security -- Injection	v1.5	5	5	5/5	Perfect
Security -- Hijack	v1.5	5	5	5/5	Perfect
Security -- Sandbox	v1.5	5	5	5/5	Perfect
Benchmark v1.0 (AXIOM)	v1.0	7	7	15.4/16	+89.5% vs raw
Benchmark v1.6 (HISTORY)	v1.6	192	192	192/192	Zero regressions

What Has Been Built -- One Line Per Milestone

01. DSL with parser, prompt store, and Streamlit UI

02. Constit

AXIOM: from blank .axiom file to a live self-governing perception agent

Built in 3 days | 100% test pass rate | +89.5% over raw model baseline